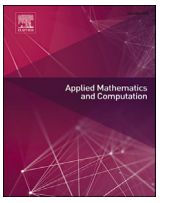
Applied Mathematics and Computation 474 (2024) 128717



Contents lists available at ScienceDirect

Applied Mathematics and Computation

journal homepage: www.elsevier.com/locate/amc



Full Length Article



Learn to solve dominating set problem with GNN and reinforcement learning ☆

Mujia Chen, Sihao Liu, Weihua He *

School of Mathematics and Statistics, Guangdong University of Technology, Guangzhou, China

ARTICLE INFO

Keywords:
Combinatorial optimization
Dominating set problem
Graph neural network
Reinforcement learning

ABSTRACT

The Dominating Set Problem has a wide range of applications in many industrial areas and the problem has been proven to be NP-hard. The idea using neural networks to solve combinatorial optimization problems has been shown to be effective and time-saving in recent years. Inspired by these studies, to solve the Dominating Set Problem, we train a neural network by Double Deep Q-Networks (DDQN). To better capture the features and structures of the graph, we use a message passing network for the graph representation. We validate our model on random graphs of different sizes, and even on several different lattice graphs, which show our model is effective.

1. Introduction

Combinatorial optimization (CO) problems are important for a wide range of computer science and industrial processes, such as operating system, scheduling, social network [18]. A valuable example of CO in power system and wireless network is *dominating set problem (DSP)*. Given a graph G, a dominating set of G is a subset D of its vertices, such that any vertex of G is either in D or has a neighbor in D. The goal of DSP is to find a dominating set with minimum cardinality. The DSP and its variants have myriad applications in many real-world domains.

1.1. Motivation

Haynes et al. [12] have proved that to find the optimal solution of DSP is NP-hard. Hence, lots of algorithms have been proposed to solve the DSP over the past decade. In fact, many algorithms to solve the DSP rely on handcrafted heuristics. But a serious weakness with this approach is that once the data of the real problem has changed slightly, the heuristic algorithm needs to search again and again, which causes a huge waste of computing resources.

Therefore, we hope that we can build some algorithms which can construct heuristics information automatically. It is well-known that deep learning is good at capturing hidden patterns of Euclidean data (images, text, videos). In contrast, learning over graphs brings numerous challenges for traditional deep learning algorithms which is effective in Euclidean data. Graph neural network (GNN) provides an easy way to transfer deep learning to graphs. Recently, extensive studies have shown that GNNs can capture hidden and complex features in various graphs even on a large scale. Thus, we consider GNNs to be an effective tool for extracting graph features and structure automatically.

☆ Supported by Guangdong Basic and Applied Basic Research Foundation (No. 2021A1515012047).

* Corresponding author.
E-mail address: hwh12@gdut.edu.cn (W. He).

https://doi.org/10.1016/j.amc.2024.128717
Received 26 December 2023; Received in revised form 25 March 2024; Accepted 28 March 2024

Available online 5 April 2024
0096-3003/© 2024 Elsevier Inc. All rights reserved.

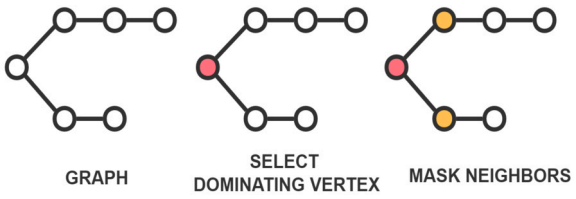


Fig. 1. Mask Processing.

Once deep learning has been used, another challenge we faced is how to train the network. Inspired by AlphaGo [21], we find that *Reinforcement Learning (RL)* has enabled algorithms to learn to make decisions without supervision. The motivation of RL is to train an agent who can try its best to get the best average return by interacting with the environment. Precisely, RL uses the experience which samples from the agent interacting with the environment, so it is widely used in unsupervised situations. This is definitely suitable for solving CO problems without enough data sets. In this paper, we focus on solving dominating set problems with deep learning methods.

1.2. Relative works

The research on DSP can be traced back to 1962 when Ore [17] first gave the definition of a dominating set. In [12,4], it has been proved that finding an exact solution to DSP is NP-hard. In the past two decades, there have been a lot of studies dedicated to finding a tighter bound for DSP. Initially, researchers tried to discuss the bounds over different graph families [7,13,23]. Particularly, Alanko et al. [1] showed the result for square grid graphs up to size 29×29. However, calculating the result on 29×29 grid graphs incredibly spent 31-CPU days.

A new efficient optimization method, called Neural Combinatorial Optimization [3], was proposed to use machine learning and deep learning model for combinatorial optimization. The earliest work which proposed the idea of using neural networks to solve CO problems can trace back to 1985 [14], which firstly was applied to the famous Travelling Salesman Problem.

Vinyals et al. [25] proposed the pointer network in 2015, which was the first encoder-decoder architecture network applied to solving CO problems. The pointer network uses the attention mechanism as a pointer to solve the problem of different output sequence lengths caused by the different sizes of training and testing graphs, which makes the model's generalization ability in the training process to the maximum extent. In addition, the pointer network was also applied to combinatorial optimization problems such as finding planar convex packets and computing Delaunay triangular profiles, all with good results. Hence, in our paper, we also use the same architecture to design our model.

With the development of graph neural networks, GNNs have also been added to encoders for feature extraction of graphs, making the feature extraction of reinforcement learning algorithms more effective. Dai et al. [15] were the first to apply graph neural networks to solve the combinatorial optimization problem.

The work by Deudon et al. [8] used the same approach as Bello et al. [3] while changing the encoder-decoder network architecture. Although the decoder part is identical to the pointer network, in this paper the GNN encoder architecture is based entirely on the Attention mechanism, so that the input is encoded as a set rather than a sequence. To address the problem of generalizing heuristic information, Chen et al. [5] proposed the NeuRewriter network, instead of directly training a solver, which learns a strategy to select a heuristic and rewrites the partial solution of the pre-training result.

2. Graph representation

DeepWalk [19], LINE [22] and Struc2Vec [20] have been used in the past to investigate graph embedding. In this paper, we use vertices heuristic information and Message-passing neural networks (MPNN) framework [10] to construct the graph representation.

2.1. Heuristic information

For DSP, we construct a one-hot vector to represent the state of the vertices of the graph. It is initialized to a $1 \times n$ zeros vector v_f, where n is the number of vertices in graph G. When a vertex i is selected, its neighbors will be dominated. Then let $v_{f_{i \cup N(i)}} = 1$ where $N(i)$ is the set of neighbors of i and v_{f_i} implies the ith position of the vector v_{f_i}. We call the above processing a *mask processing* and call the one-hot vector v_{f_i} *vertex features*. The processing can be shown as the Fig. 1.

2.2. Graph embeddings

MPNN framework is quite robust as we can define different messages and update functions depending on what we want to achieve [10]. For each vertex v_i of the graph, we aggregate a function of its hidden states, hidden states of neighbors, and features of possible edges of all neighboring vertices.

$$m_v^{t+1} = \sum_{w \in N(v)} M_t(h_v^t, h_w^t, e_{vw}), \tag{1}$$

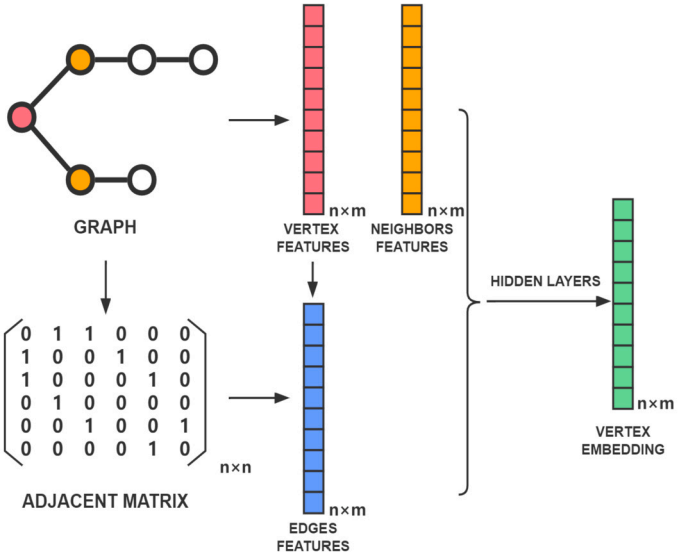


Fig. 2. Vertices Embedding.

where m is the embedding of vertex v in $t+1$ layer $(n \times 3n_f)$, h is the hidden states $(n \times n_f)$ and M is the aggregate function. To reinforce the embedding, linear layers and activation functions are used as (2). In this paper, we use a 3-layers MPNN to capture vertices embedding

$$\hat{m_v^{t+1}} = ReLU(\theta_2 ReLU(\theta_1 m_v^{t+1})), \tag{2}$$

where θ_1 and θ_2 are parameters of linear layers. $\hat{m_v^{t+1}}$ is the final vertex embedding $(n \times n_f)$. And we get the edges embedding

$$e = ReLU(\theta_3 CONCAT(f(v_f, A), A_{norm})), \tag{3}$$

where e is the edges embedding $(n \times n_f)$. A_{norm} is the normalization of A and

$$f = ReLU(\theta_4 CONCAT(v_f, A)), \tag{4}$$

where A is the adjacency matrix of graph G and $CONCAT$ means a concat function. The processing of vertices embedding is shown in Fig. 2.

3. Reinforcement learning

Reinforcement learning (RL) has been widely used to solve the CO problem. We define the states, actions, and rewards and use *Double Deep Q network (DDQN)* to train our model.

3.1. RL formulation

1. *States*: A state S is a one-hot vertex features vector v_f. The state vector will be combined with other graph features to get the embedding through the neural network as mentioned in Section 2.
2. *Actions*: An action is a vertex of graph G. When action v is taken, itself and its neighbors will be masked and the state vector v_f will be updated. Then we can get an action or a Q value through the neural network. When a new vertex is selected, the state will be updated by a *mask processing*. It means that when a vertex v is selected, $u \in (v \cup N(v))$ can not be selected next time since neighbors of v have been dominated.
3. *Rewards*: The reward is defined as the change in the number of dominated vertices. It can be calculated as (5), where α is a rate to adjust reward usually set 1 or 2, $A^{(i)}(1 \times n)$ is the ith column of adjacent matrix $(n \times n$ and $\omega(1 \times n)$ is the vertex weight vector. $\hat{v_f}(1 \times n)$ is a $1, -1$ vector deformed from v_f, $\hat{v_{f_i}} = -1$ where $v_{f_i} = 1$ and $\hat{v_{f_i}} = 1$ where $v_{f_i} = 0$. The symbol $\odot$ means the *Hadamar Product*. In other words, if a selected vertex can dominate more new vertices, the agent will get a positive reward. In the contrast, if a selected vertex is adjacent to more vertices that have been dominated, then the agent will get a negative reward.

$$R(S, v_i) = \alpha \hat{v_f}^T (A^{(i)} \odot \omega). \tag{5}$$

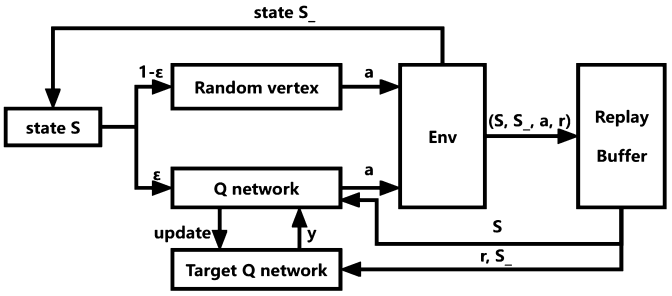


Fig. 3. DDQN Processing.

4. *Evaluate Function* $Q(h(S), v; \theta)$: to summarize the state of a graph G and figure out the value of a new vertex if it is to be selected as a new dominating vertex. In this paper, we use a function Q to evaluate the value of an action, where $h(S)$ is the hidden features of the state S which get from the vertices embedding and v is the selected vertex. After T iterations, Q $(1 \times n)$ can be calculated as (6) and the parameter Θ will be learned through *DDQN*.

$$Q(h(S), v; \Theta) = \theta_5^T ReLU(\theta_6 \sum_{u \in V} \hat{m_u^T}, \theta_7 \hat{m_v^T}). \tag{6}$$

5. *Policy*: a greedy policy π based on $Q(h(S), v)$ will be used as (7).

$$\pi(v \mid S) := argmax_{v' \in \bar{S}} Q(h(S), v'), \tag{7}$$

where $\bar{S}$ is $V - S$.

3.2. DDQN

*D*ouble Deep Q Network (DDQN) is an improvement over *Deep Q Network* (DQN). *DQN* uses a neural network to replace the *Q*-table in *Q-Learning*, it saves the storage space and can solve both continuous state space and discrete action space problems. *DQN* has been widely used in many areas such as path planning [16], power control [6]. However, *DQN* has some weaknesses. First, bootstrapping will cause the transmission of deviation. Second, maximization causes the overestimation of TD targets.

In *DQN*, TD targets at time t is

$$y_t = r_t + \gamma \max_{v \in V} Q(h(s_{t+1}), v; \Theta),$$

but in *DDQN* it is

$$\hat{y}_t = r_t + \gamma Q(h(s_{t+1}), v^*; \bar{\Theta}),$$

where $v^* = \arg\max_{v \in V} Q(h(s_{t+1}), v; \Theta)$ and $\bar{\Theta}$ is the parameter of value target network. It can be proved that $\hat{y}_t < y_t$. In other words, *DDQN* improve the overestimation [24].

The *DDQN* processing is presented as in Fig. 3. To use *DDQN*, we firstly calculate q value at time t: $\hat{q_t} = Q(h(s_t), v_t; \Theta_{now})$, then get a^* and calculate q value at time $t+1$: $\hat{q_{t+1}} = Q(h(s_{t+1}), v^*; \bar{\Theta_{now}})$. We can get TD target $\hat{y} = r_t + \gamma \hat{q_{t+1}}$ and TD error $\delta_t = \hat{q_t} - \hat{y_t}$. In this paper, we set *MSE* loss function to get the loss

$$Loss = \frac{1}{2}(\hat{q_t} - \hat{y_t})^2. \tag{8}$$

Using the loss to backward and get the gradient $\bigtriangledown_\theta Q(s_t, v_t; \theta_{now})$. Then making gradient descent to update the q value neural network and target value neural network

$$\Theta_{new} \leftarrow \Theta_{now} - \beta \cdot \delta_t \cdot \bigtriangledown_\theta Q(h(s_t), v_t; \theta_{now}),$$
$$\bar{\Theta}_{new} \leftarrow \tau \cdot \Theta_{new} + (1 - \tau) \cdot \bar{\Theta}_{now},$$

where β is the learning rate and τ is the updating rate of target value neural network. The whole RL processing shows as followed.

4. Experiments

4.1. Instance generation

To train and evaluate the proposed embedding method and RL model, we generate instances for *DSP*. In this paper, we generate Erdös-Renyi (ER) [9] graphs and Barabasi-Albert (BA) [2] graphs which have been used in many models evaluation [15]. And to make a further evaluation, we generate grid graphs, triangular lattice graphs, and hexagonal graphs.

Algorithm 1 DSP Reinforcement Learning Processing.

Initialize neural network
Initialize experience replay buffer M to capacity to N.
Initialize environment and get an initial state s.
Set training start when $t > n$. Set $done = True$. Set update frequency of target neural network p.
for t to T do **do**
  while done **do**
    $$v_t = \begin{cases} \text{random} \quad v \quad from \quad \bar{S}, & w.p. \quad \epsilon \\ \arg\max_{v \in \bar{S}} \hat{Q}(h(s_t), v; \Theta), & otherwise \end{cases}$$
    $s_{t+1}, r, done := env.step(s_t, v_t)$
    if $M.len < N$ **then**
      Add tuple (s_t, s_{t+1}, v_t, r) to replay buffer M
    end if
    $s_t = s_{t+1}$
  end while
  Reset the environment to get a new graph.
  if $t > n$ **then**
    Sample batch from experience replay buffer
    Update Θ by Adam.
  end if
  if $t\%p == 0$ **then**
    Update $\bar{\Theta}_{new} \leftarrow \tau \cdot \Theta_{new} + (1 - \tau) \cdot \bar{\Theta}_{now}$
  end if
end for

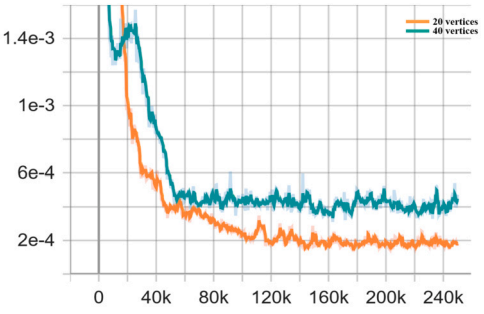


Fig. 4. Loss of model training on graphs with 20 and 40 vertices.

4.2. Experimental design

1. **Training**: we train the proposed model on ER random graph $G(n, p)$, where n is the number of vertices of G and p is the probability of connection. We set $n = 20, 40, 80$, and 100 respectively, and set $p = 0.4$. During the training processing, we use another stable set of random graphs to test the model and get test scores to evaluate the model.
2. **Evaluation**: firstly, we evaluate the model on ER graph $G(n, p)$ and BA graph $G(n, m)$ (set $p = 0.8$ and $m = 18$ in order to make graph network complex). We set $n = 20, 40, 100, 200, 300, 400$, and 500 respectively to test the generalization ability of the model and we evaluate the model by comparing it with the greedy algorithm. If the solution of the model is better than the greedy algorithm or costs less time than it is effective. Secondly, we also evaluate the model on grid graphs, triangular lattice graphs, and hexagonal lattice graphs. Finally, we compare the solution of the proposed model with the optimal solution of DSP. If our model gets an optimal solution or gets a solution that approximates the optimal solution and costs less time than the proposed model is effective and performs well.

4.3. Results display

4.3.1. Training

Fig. 4 and Fig. 5 show the loss of the model training on graphs with $20, 40, 80$, and 100 vertices. We set learning rate $\beta = 1 \times 10^{-4}$ and set $\epsilon = 0.05 \sim 1.00$. The exploration degree increase with the increase of the number of training time steps. The size of the experience replay buffer is set to 500. The main model updates every 32 episodes and the target model updates 1000 episodes. The reward discount rate is set as 1.00.

For graphs with 20 and 40 vertices, we train a model for 200000 episodes. And for graphs with 80 and 100 vertices, we train the model for 800000 episodes. The final loss of the model trained on graph with 80 and 100 vertices is less. And the loss of our models trained on different graphs all converge, which implies the robustness and correctness of the model.

Fig. 6, Fig. 7, Fig. 8 and Fig. 9 show the scores of model training on graphs with $20, 40, 80$, and 100 vertices. During the training of our model, we evaluate models by testing their performance on some new random graphs. For graphs with 20 and 40 vertices, we

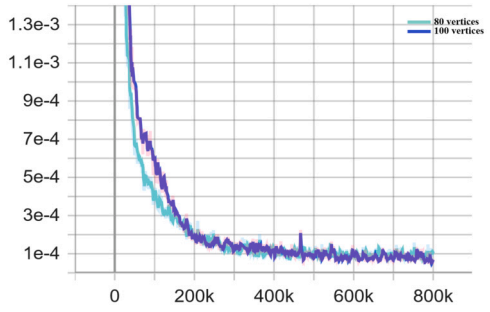


Fig. 5. Loss of model training on graphs with 80 and 100 vertices.

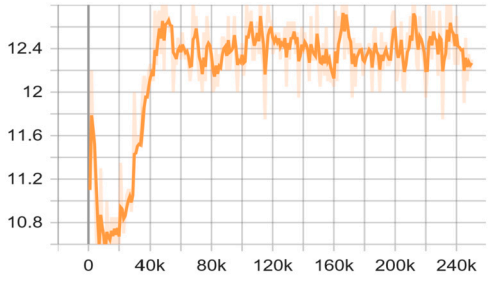


Fig. 6. Scores of model training on graphs with 20 vertices.

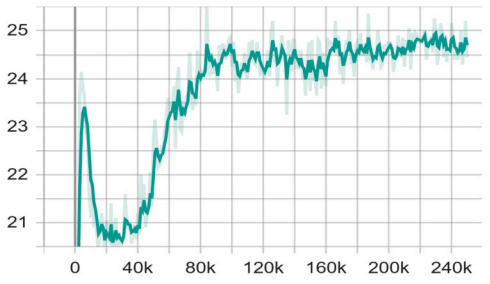


Fig. 7. Scores of model training on graphs with 40 vertices.

evaluate the model every 1000 time steps on the random graph to get a score. And for graphs with 80 and 100 vertices, we evaluate the model every 3200-time steps. Once we evaluate the model, we select some graphs from the set of random graphs and use the model the get the solution of DSP. Then we get the average scores of evaluation on random graphs. If the score is higher, then the model is getting better.

As the figures show, the score of the model rises firstly and then converges, which implies the model is effective. It should be noted that there exists a high point at the beginning of the score figure, it expresses the random initialization of neural networks and the random exploration of agents. At the beginning of the training, the agent has no experience and explores randomly. As the training progresses, the agent learns the best strategy for picking a vertex to improve the score of the model. That is although the model get a relatively high score at the beginning since the initialization, the agent does not learn anything. Therefore, scores will decrease at first and then increase and finally converge.

4.3.2. Evaluation

Table 1 shows the results of the evaluation of our model trained on 20 vertices, which compare with the results of using the greedy algorithm. It shows that our model performs better than the greedy algorithm, especially on large BA graphs. And on ER graphs, our model is significantly more efficient than the greedy algorithm according to the calculation time.

Table 2 shows the results of the evaluation of our model trained on 20 and 100 vertices. It shows that the proposed model performs better than the greedy algorithm, especially on triangular lattice graphs. The proposed model can get a much smaller dominating number on grid graphs, triangular lattice graphs, and hexagonal lattice graphs. We show some solutions of DSP on three kinds of graphs as Fig. 10, Fig. 11 and Fig. 12.

According to the results of Gonçalves et al. [11], the domination number of grid graphs $G_{m,n}$ is equal to $\lfloor \frac{(n+2)(m+2)}{5} \rfloor - 4$, where $m, n \geq 16$. Considering the graphs with 300 (15×20), 400 (20×20 and 500 (20×25) vertices, their exact dominating number are $70, 92$, and 114, respectively. The dominating numbers solved by our models are $83, 107$, and 136, respectively. The dominating

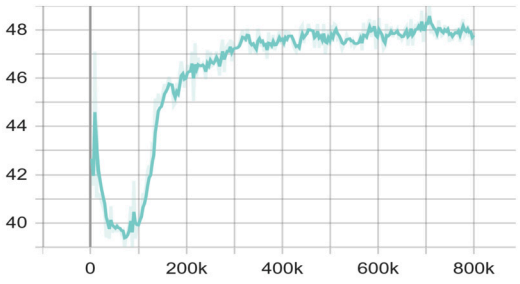


Fig. 8. Scores of model training on graphs with 80 vertices.

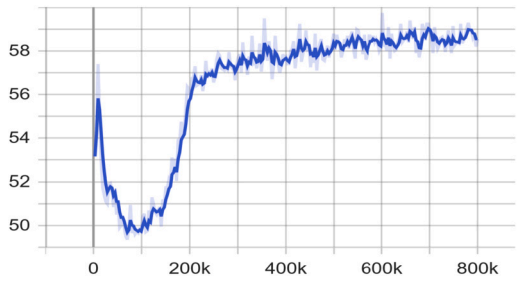


Fig. 9. Scores of model training on graphs with 100 vertices.

Table 1
Evaluation of proposed model on BA and ER graphs.

	MODEL SCORES	GREEDY SCORES	MODEL TIME/s	GREEDY TIME/s
BA_20	19	19	0.00662	0.00072
BA_40	36.93	36.94	0.008407	0.00086
BA_80	**59.6**	59.59	0.026392	0.00168
BA_100	68.66	68.67	0.040887	0.002634
BA_200	**110.11**	109.75	0.085545	0.008331
BA_300	**148.57**	147.23	0.184033	0.020803
BA_400	**186.82**	183.06	0.735913	0.21928
BA_500	**224.4**	218.54	1.207629	0.392946
ER_20	18	18	0.006333	0.00063
ER_40	36.18	36.18	0.00705	0.000766
ER_80	71.28	71.28	0.009807	0.00096
ER_100	88.44	88.44	0.010721	0.001062
ER_200	173.56	173.56	**0.048772**	0.076701
ER_300	258.14	258.14	**0.099572**	0.175197
ER_400	342.04	342.04	**0.165747**	1.592013
ER_500	424.77	424.77	**0.211523**	3.092594

1 The above model is trained on graphs with 20 vertices.
2 The GPU is GTX1050, it means that the cost time can be smaller.

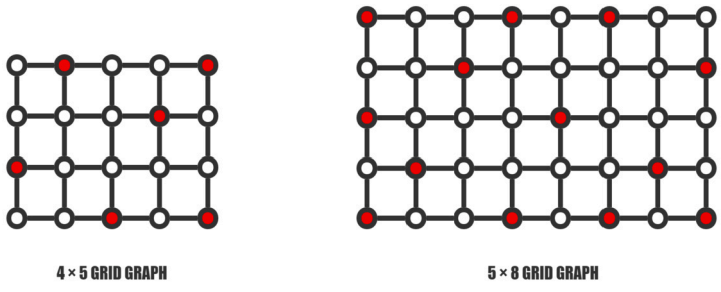


Fig. 10. Solution of DSP on grid graphs.

Table 2
Evaluation of proposed model on three types of graphs.

GRID			TRI			HEX		
n	γ	γ_g	n	γ	γ_g	n	γ	γ_g
MODEL TRAINED ON 20 VERTICES GRAPH								
20	**6**	8	20	**4**	6	22	6	7
36	**10**	13	40	**9**	10	38	11	11
63	**19**	21	60	**12**	17	58	17	17
81	**23**	27	78	**16**	22	82	23	23
100	**27**	33	102	**23**	28	96	**26**	28
200	**55**	63	200	**43**	49	196	**52**	57
300	**84**	94	305	**57**	77	306	**81**	89
400	**107**	124	406	**74**	99	418	**109**	115
500	**137**	154	496	**87**	121	510	**132**	140
MODEL TRAINED ON 100 VERTICES GRAPH								
100	**29**	33	102	**23**	28	96	29	28
200	**59**	63	200	**40**	49	196	57	57
300	**83**	94	305	**57**	77	306	**87**	89
400	**113**	124	406	**73**	99	418	**111**	115
500	**136**	154	496	**88**	121	510	**135**	140

1 GRID means the grid graph, TRI means the triangular lattice graph and HEX means the hexagonal lattice graph.
2 n is the number of vertices of the graphs.
3 γ is the dominating number of graphs get by the DSP-RL model and γ_g is the dominating number of graphs get by the greedy algorithm.

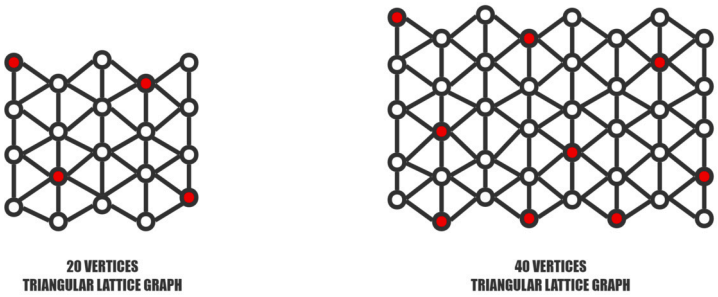


Fig. 11. Solution of DSP on triangular lattice graphs.

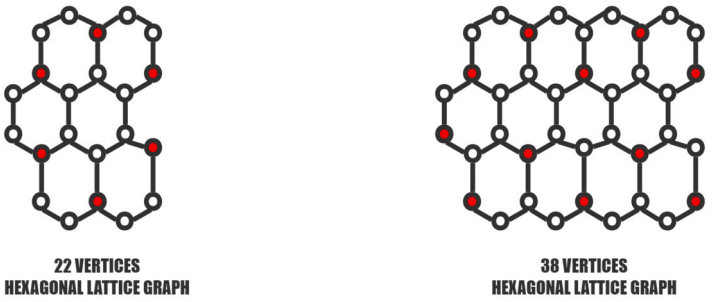


Fig. 12. Solution of DSP on hexagonal lattice graphs.

numbers solved by the greedy algorithm are 94, 124, and 154, respectively. It shows our model performs much better than the greedy algorithm. And the error rate between our model and the exact solution can be as low as 16.3%.

Alanko et al. [1] have got the optimal dominating number of grid graphs $G(m,n)$ where $m,n \leq 29$. We make a further evaluation by comparing our solution with the optimal solution on grid graphs where $5 \leq m \leq 10,\ 4 \leq n \leq 10$. Table 3 shows the results of the evaluation. γ is the dominating number of graphs get by the proposed model, γ_g is the dominating number of graphs get by greedy algorithm and γ_* is the optimal dominating number. $\Delta = \gamma - \gamma_*$ and $\Delta_g = \gamma_g - \gamma_*$.

As the results show, our model performs well on DSP. On $G(5,4)$ and $G(6,6)$ the model can get the optimal solution. And the average error of dominating number on grid graphs $G(m,n)$ where $m,n \leq 29$ is 2, but using the greedy algorithm is 4. And according to Alanko et al. [1], they cost 31 CPU-days (using a 3-GHz Intel Core2 Duo CPU E8400) for calculating the dominating number of

Table 3
Optimal solution versus solution of proposed model and greedy algorithm on grid graphs $G(m,n)$.

m	n	γ	γ_g	γ^*	Δ	Δ_g
5	4	**6**	8	6	0	2
5	5	8	8	7	1	1
5	6	10	11	8	2	3
5	7	11	13	9	2	4
5	8	13	13	11	2	2
5	9	13	16	12	1	4
5	10	15	18	13	2	5
6	6	**10**	13	10	0	3
6	7	12	14	11	1	3
6	8	13	16	12	1	4
6	9	16	17	14	2	3
6	10	17	20	16	1	4
7	7	17	17	12	5	5
7	8	19	19	14	5	5
7	9	19	21	16	3	5
7	10	19	23	17	2	6
8	8	18	20	16	2	4
8	9	22	22	18	4	4
8	10	23	26	20	3	6
9	9	23	27	20	3	7
9	10	25	29	24	1	5
10	10	27	33	24	3	9

grid graph $G(29,29)$. However, the proposed model only uses 9.7744s (using an Intel Core i5-7300HQ CPU and a GPU GTX1050). It shows the model cost much less time.

In other words, our model performs much better than the greedy algorithm and it can get optimal solution of DSP in some situations which implies that the model is effective.

5. Conclusion

This paper proposes a model based on reinforcement learning and graph neural networks to solve DSP. Firstly we use a neural network framework base on MPNN to get the graph embedding. Then we use reinforcement learning to solve the DSP. We design experiments and train the model in several different graphs. Finally, we evaluate the proposed model by comparing it with the greedy algorithm and some existing results.

The results of experiments show that the model is effective. The model gets a better solution of DSP on BA graphs, especially on large BA graphs, and costs less time on ER graphs. And the model performs well on grid graphs, triangular lattice graphs, and hexagonal graphs. It can get a much better solution than the greedy algorithm.

Data availability

Data will be made available on request.

References

[1] Samu Alanko, Simon Crevals, Anton Isopoussu, Patric Östergård, Ville Pettersson, Computing the domination number of grid graphs, Electron. J. Comb. 18 (1) (2011) [electronic only], Research–Paper.
[2] Albert Réka, Albert-László Barabási, Statistical mechanics of complex networks, Rev. Mod. Phys. 74 (1) (2002) 47.
[3] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, Samy Bengio, Neural combinatorial optimization with reinforcement learning, in: ICLR2017, 2017.
[4] John Adrian Bondy, Uppaluri Siva Ramachandra Murty, et al., Graph Theory with Applications, vol. 290, Macmillan, London, 1976.
[5] Xinyun Chen, Yuandong Tian, Learning to perform local rewriting for combinatorial optimization, Adv. Neural Inf. Process. Syst. 32 (2019).
[6] Ye Chen, Yanda Li, Dongjin Xu, Liang Xiao, Dqn-based power control for iot transmission against jamming, in: 2018 IEEE 87th Vehicular Technology Conference (VTC Spring), IEEE, 2018, pp. 1–5.
[7] Ernest J. Cockayne, B. Gamble, Bruce Shepherd, An upper bound for the k-domination number of a graph, J. Graph Theory 9 (4) (1985) 533–534.
[8] Michel Deudon, Pierre Cournut, Alexandre Lacoste, Yossiri Adulyasak, Louis-Martin Rousseau, Learning heuristics for the tsp by policy gradient, in: International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research, Springer, 2018, pp. 170–181.
[9] Paul Erdős, Alfréd Rényi, et al., On the evolution of random graphs, Publ. Math. Inst. Hung. Acad. Sci. 5 (1) (1960) 17–60.
[10] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, George E. Dahl, Neural message passing for quantum chemistry, in: International Conference on Machine Learning, PMLR, 2017, pp. 1263–1272.
[11] Daniel Gonçalves, Alexandre Pinlou, Michaël Rao, Stéphan Thomassé, The domination number of grids, SIAM J. Discrete Math. 25 (3) (2011) 1443–1453.
[12] Teresa W. Haynes, Stephen Hedetniemi, Peter Slater, Fundamentals of Domination in Graphs, CRC Press, 2013.
[13] W. Teresa, Haynes, Stephen T. Hedetniemi, Michael A. Henning, Topics in Domination in Graphs, vol. 64, Springer, 2020.
[14] John J. Hopfield, David W. Tank, "Neural" computation of decisions in optimization problems, Biol. Cybern. 52 (3) (1985) 141–152.

[15] Elias Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, Le Song, Learning combinatorial optimization algorithms over graphs, Adv. Neural Inf. Process. Syst. 30 (2017).
[16] Liangheng Lv, Sunjie Zhang, Derui Ding, Yongxiong Wang, Path planning via an improved dqn-based learning policy, IEEE Access 7 (2019) 67319–67330.
[17] O. Ore, Theory of Graphs, American Mathematical Society Colloquium Publications, American Mathematical Society, 1962.
[18] Vangelis Th Paschos, Applications of Combinatorial Optimization, vol. 3, John Wiley & Sons, 2014.
[19] Bryan Perozzi, Rami Al-Rfou, Steven Skiena Deepwalk, Online learning of social representations, in: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2014, pp. 701–710.
[20] Leonardo FR Ribeiro, Pedro H.P. Saverese, Daniel R. Figueiredo, Struc2vec: learning node representations from structural identity, in: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2017, pp. 385–394.
[21] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al., Mastering the game of go with deep neural networks and tree search, Nature 529 (7587) (2016) 484–489.
[22] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, Qiaozhu Mei Line, Large-scale information network embedding, in: Proceedings of the 24th International Conference on World Wide Web, 2015, pp. 1067–1077.
[23] Jennifer M. Tarr, Domination in Graphs, Graduate Theses and Dissertations, USF, Tampa, 2010.
[24] Hado Van Hasselt, Arthur Guez, David Silver, Deep reinforcement learning with double q-learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 30, 2016.
[25] Oriol Vinyals, Meire Fortunato, Navdeep Jaitly, Pointer networks, Adv. Neural Inf. Process. Syst. 28 (2015).